

# Assignment 4: Refactoring

## 1: Long Method

### Commit 27f802e

Within `Player.updateInternal`, the method alone ran lots of code, including the stun handling, input handling, and movement.

This was refactored to split off into an input handling method, which then calls to a move method.

The stun may further be split into it's own method later.

## 2: Primitive Obsession

### Commit 2cd2e4c

Within `MobileEntity`, the parent abstracts a `getAttackRange` method which is continuously drawn from by the abstracted part.

While needed, this method was just used as a holder for a primitive value which is only used in the parent.

Instead, this was changed such that the abstracted counterpart sets the variable for the attack range upon creation such that no internal calls are needed to obtain this constant data.

## 3: Inappropriate Intimacy

### Commit eb2a767

For world initialization, the worldloader directly injected entities into the world entity vector, and manually set the player pointer, which couples these classes way too tightly.

This was refactored using existing `addEntity` logic used within testing, which now handles all entity adding done within the worldloader.

## 4: God Class

### Commit 65fbfbc

The `GameWorld` class ran way too much code, and was responsible for almost everything.

To alleviate this, the physics was instead refactored into it's own singleton class responsible for handling `solidObjects`, and the `requestMove` logic.

## **5: Duplicated Code**

### **Commit d846a5e**

In the MobileEnemy class, the logic of determining the enemy's attack range was duplicated in several ways in the isPlayerWithinAttackRange and tryAttackPlayer methods. A helper method getEdgeDistanceToPlayer was created that solved the problem of calculating the distance from the edges of the enemy's and player's hitboxes. This eliminated code duplication and made the logic clearer for future updates.

## **6: Duplicated Code**

### **Commit 71d930d**

In the MobileEnemy class, the chase and patrol path rebuilding logic used the same setup code, which included checking the map for validity, getting the player's current position, and the general stage of rebuilding the path. This is a duplication of the complexity in maintaining the code, since it was necessary to change the code in two places. The getValidTileMap, getCurrentTilePosition, and rebuildPathToTarget methods were created to perform these tasks. This improved readability and reduced the amount of repeated code.

## **7: Long Method**

### **Commit c56fa85**

In the MobileEnemy class, the updateInternal method performed too many functions independent of each other, in particular, updating the movement state, deciding whether to rebuild path, and following the path. This increased the time to understand the functions of the method. This problem was solved by extracting the helper methods handlePathRebuild and followCurrentPath. As a result, the state update method has become shorter and more readable.

## **8: Long Method**

### **Commit 4502467**

In the Eraser class, the beforeMovementUpdate method performed several unrelated tasks, such as adjusting the position when appearing, updating the cooldown, checking movement, updating the side of the surface to correctly draw the sprite, and erasing graphite from the floor. This made it more difficult to use the method. This method was divided into subtask methods that were each responsible for their own function. This made the structure more logical.